BUILD WHAT PAYS

A SOLO BUILDER'S FIELD MANUAL FOR FINDING IDEAS THAT PAY

Michael Colenso

Build What Pays

Copyright © 2026 Michael Colenso
All rights reserved.
First Edition, 2026

Why This Book

I did not build this framework because I wanted a cleaner way to brainstorm.

I built it because I got tired of watching smart builders waste time on weak bets.

My working life has been shaped by operations, sequencing, constraints, and consequence. I spent more than twenty years in construction project management, where vague thinking gets expensive fast. On a real project, you do not get points for interesting ideas. You get measured on whether the thing works, whether the timing holds, whether the budget survives contact with reality, and whether the risks were visible early enough to matter. That operating instinct came with me into software, automation, data products, and AI-assisted builds.

The pattern showed up immediately.

Builders were not short on ideas. They were drowning in them. Every trend produced another wave of “maybe this could be a startup” energy. Every new model or tool created another excuse to go hunting for a problem after the solution already felt exciting. Every founder thread became another reminder that the market rewards judgment more than ideation, but almost nobody was building systems for judgment.

So I started scoring opportunities harder.

Not in a theoretical way. In a practical, repetitive, operator way. Who is the buyer? How painful is the problem? Is money already moving? Is the trigger obvious? Can the first version ship without drowning in complexity? Is there a believable path to distribution, or am I lying to myself because the build sounds fun?

I have evaluated more than two hundred opportunities this way. Most of them failed the filters.

That is the point.

A good framework should reject more ideas than it advances. If it blesses everything, it is not a framework. It is a mood board.

This book came out of those repetitions. It is a compressed system built from seeing the same failure modes over and over: novelty bias, vague buyers, no trigger, no path to money, too much software before proof, and founders mistaking their own excitement for market reality.

If you use this book well, it will probably save you from building a few things you currently like. Good. That is cheaper than building them.

Contents

Introduction

Most builders do not fail because they cannot execute. They fail because they choose weak ideas, validate too late, and confuse interest with demand.[1]

They mistake motion for evidence. They generate long lists of plausible products, follow whatever is trending, or fall in love with a tool before they have earned a problem. Then they spend weeks or months building something that never had enough pain, urgency, or spending behind it to matter.

That is the trap.

The market leaks signal everywhere. Operators complain about ugly workflows in forums. Small businesses pay monthly for software they resent. Public data sits in broken interfaces with zero packaging and obvious commercial value. Buyers reveal intent through permits, licenses, inspections, filings, violations, complaints, recalls, and other operational signals long before most founders notice.

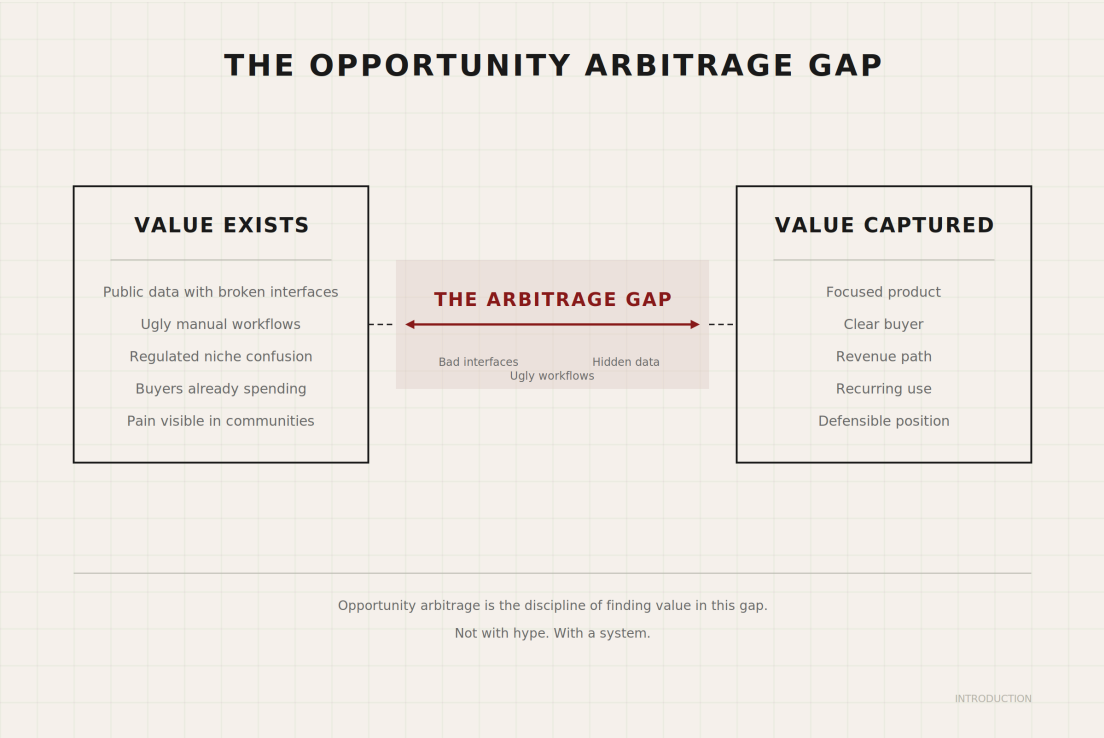
Opportunity arbitrage is the discipline of finding value in that gap.[2]

Not with hype. Not with generic “AI startup ideas.” Not with another brainstorm dump.

With a system.

[1] The most common failure mode is not execution failure. It is selection failure.

[2] **Opportunity arbitrage:** finding markets where value already exists but is poorly surfaced, poorly packaged, or poorly served.



A strong solo-builder opportunity usually has some combination of these traits:

- a clear buyer
- a painful, repeated problem
- visible money already changing hands
- a narrow version that can ship fast
- a believable path to first revenue
- enough defensibility to matter, without so much complexity that the build collapses under its own ambition

The goal is not to find the most glamorous idea. The goal is to find the strongest wedge.[3]

This book will help you do three things: find higher-signal opportunities, kill weak ideas faster, and validate before you overbuild.

If this book works, it should make you more ruthless. Ruthless about what counts as evidence. Ruthless about what gets cut. Ruthless about what deserves your next 30 days.

[3]**Wedge:** the narrowest, most specific version of a product that can prove demand. Not the whole vision. The entry point.

CHAPTER 1

The Idea Trap

Most builders think their job starts with ideas. It does not. It starts with filters.[4]

An unfiltered idea pipeline produces junk. You can make almost anything sound plausible if you stay abstract enough: a marketplace for X, an AI assistant for Y, a dashboard for Z. Plausibility is cheap. The market does not pay for plausibility.

Weak ideas usually survive for the same reasons.

Novelty Bias

Builders chase what feels new, not what is painful. A clever product gets more attention in founder circles than a boring product that solves a recurring operational headache.

Proximity Bias

Builders overvalue problems they personally find interesting, even when the buyer is unclear or the budget is weak.

Tool-First Thinking

AI, scraping, automation, agents, and workflows become excuses to go looking for a problem after the solution already feels exciting.[5]

Compliment Confusion

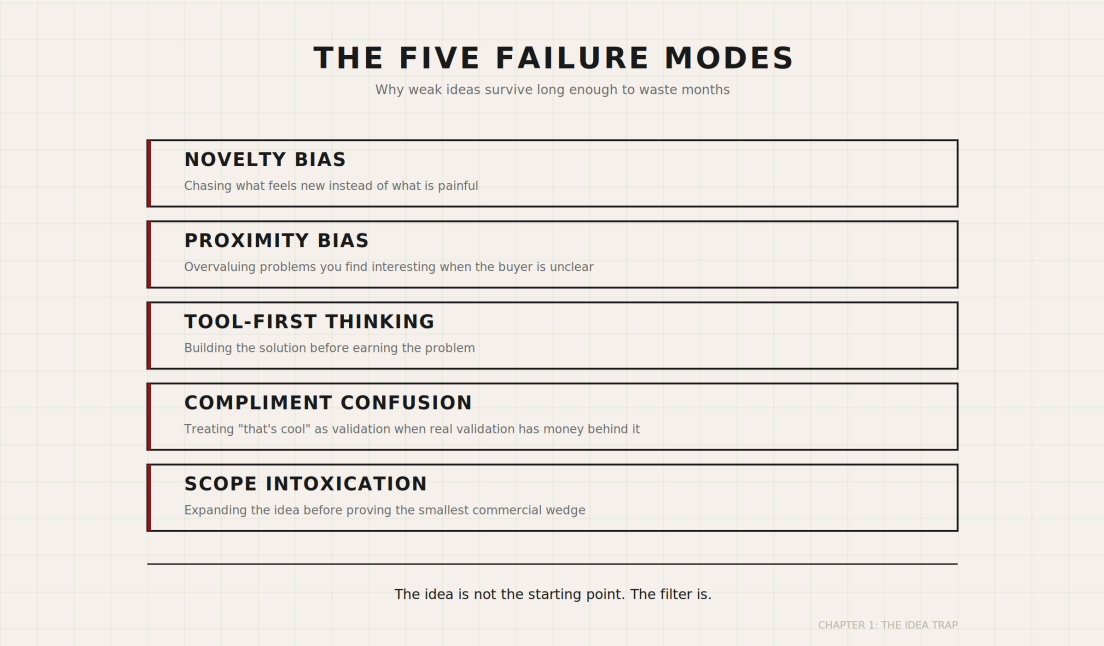
People say, “That’s cool,” or, “I’d use that,” and founders treat it like validation. It is not. Real validation has cost, effort, urgency, or money behind it.

Scope Intoxication

The idea sounds big enough to matter, so the founder expands it before proving the smallest commercial wedge.

[4] Good founders are not idea generators. They are idea editors.

[5] If the best argument for the idea is that the stack is interesting, the market is probably not the real driver.



The result is predictable: months of work, weak demand, and a stack of explanations for why the market just was not ready.

Good founders are not just idea generators. They are idea editors. They know how to cut.

CHAPTER 2

The Money-First Lens

A money-first lens does not mean greed-first. It means evidence-first.[6]

The strongest early signal is not, “People care about this topic.” It is, “People are already paying, losing money, or burning serious time because this problem is unresolved.”

There are three especially strong signals.

Current Spending

If people already pay for an ugly tool, a manual service, a consultant, an attorney, or a spreadsheet-heavy process, there is usually room to build something sharper.

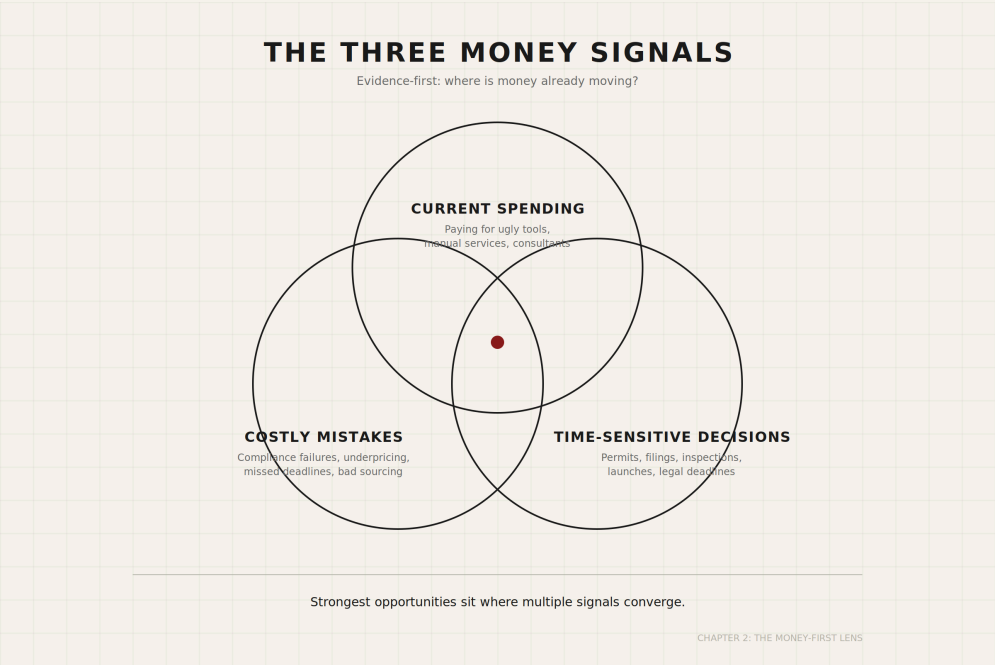
Costly Mistakes

Markets with repeated errors, compliance failures, underpricing, missed deadlines, bad sourcing, or weak tracking create stronger purchase intent than markets full of casual wish lists.

Time-Sensitive Decisions

If a buyer needs the answer now because of a permit, launch, filing, inspection, deadline, or legal risk, urgency compresses the sales cycle.[7]

[6] **Money-first ≠ greed-first.** It means: where is evidence of spending, loss, or time-burn already visible?
[7] Urgency is the best sales accelerant. Permits, inspections, and compliance deadlines create it naturally.



A money-first builder asks: What are they already paying for? What mistakes are currently costing them money? What ugly process are they tolerating because nothing better exists? Can software, AI, or automation collapse this pain enough to matter?

That is why small, specific markets so often beat broad inspirational categories. A reseller trying to survive tax season, a contractor trying to catch permits before competitors, or a food founder trying to launch legally has a stronger buying trigger than someone vaguely asking for better productivity.

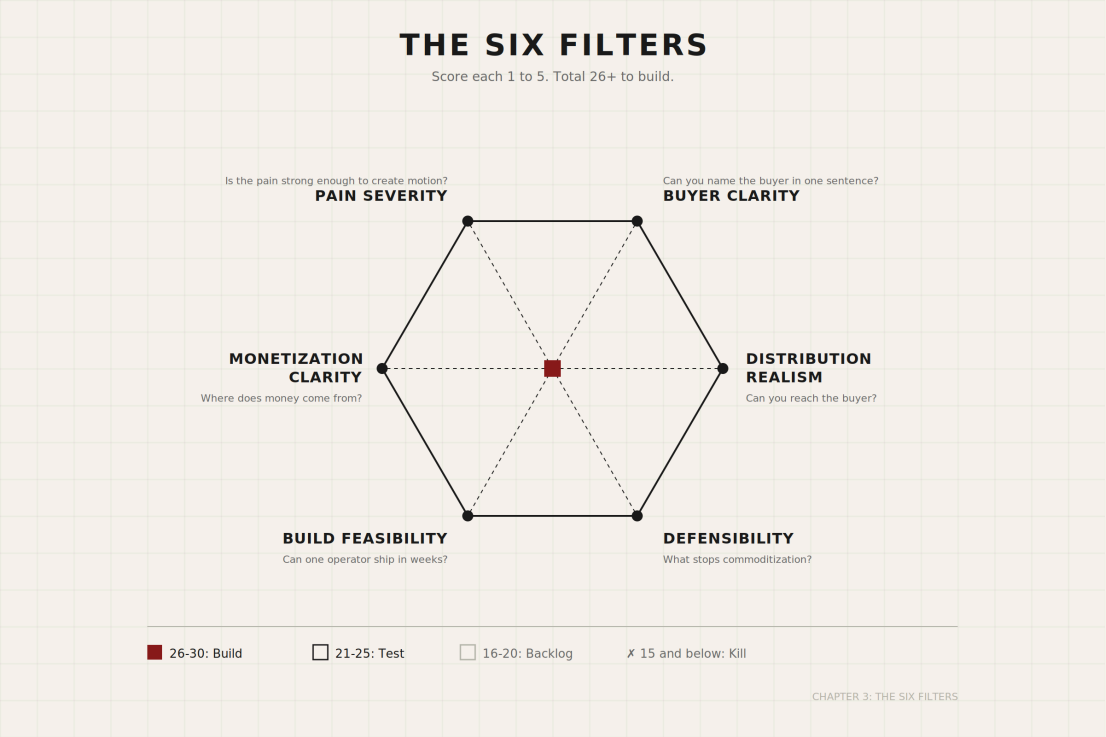
Specific pain creates specific products.

CHAPTER 3

The Six Filters

Most idea evaluation is too soft. Founders say things like “big market,” “strong trend,” or “interesting space.” None of that is enough.

Use these six filters on every opportunity before you let yourself like it.[8]



Buyer Clarity

Can you name the buyer in one sentence?[9]

Not “small businesses.” Not “creators.” Not “anyone who needs help with X.”

A useful buyer definition sounds like this: first-time hot sauce entrepreneur trying to sell legally; part-time reseller with tax-season spreadsheet chaos; leather crafter underpricing custom commissions; roofer who wants permit-triggered sales alerts in one county.

[8] Score each filter 1–5. Total of 26+ = build candidate. 15 or below = kill.

[9] Test: “first-time hot sauce entrepreneur trying to sell legally” passes. “Small businesses” does not.

If the buyer is fuzzy, the product will be fuzzy.

Pain Severity

Is the problem painful enough to create motion?

Strong pain usually costs money, creates legal or compliance risk, wastes significant time, causes repeated operational mistakes, blocks revenue, or creates public embarrassment. A mild inconvenience rarely converts. A recurring operational wound often does.

Monetization Clarity

How does this make money quickly and believably?[10]

There should be a visible path to subscription, lead generation, affiliate revenue, premium alerts, service revenue before software, or a narrow B2B tool with obvious ROI.

Build Feasibility

Can one operator get to proof without dying in complexity? The question is not whether the idea can be built eventually. Nearly anything can. The question is whether the smallest wedge can ship in weeks, not quarters.

Defensibility

If this works, what stops it from becoming a commodity? Early defensibility does not need to mean a giant moat. It means there is something that gets harder to copy over time: curated data, workflow fit, niche trust, domain-specific scoring, or recurring operational use.

Distribution Realism

Do you have a believable path to the buyer?[11] A strong idea with weak access can still be a bad bet. If the distribution thesis is basically hope, the opportunity is not ready.

The point is not false precision. The point is forced judgment.

[10] If the money story requires a paragraph of explanation, the idea is weaker than it feels.

[11] A strong idea with weak distribution is still a bad bet. Hope is not a channel.

CHAPTER 4

Three Discovery Engines

Good opportunities tend to emerge from the same few mechanics. Once you understand the mechanics, you stop waiting for inspiration.[12]

Engine 1: Money Trails in Communities

Look where people openly discuss what they pay for, what tools they hate, what mistakes cost them money, what manual work they are tired of doing, and what services they wish were better or cheaper.

Good hunting grounds include subreddits, niche forums, Facebook groups, review sites for ugly vertical software, and practitioner-heavy YouTube comment sections.

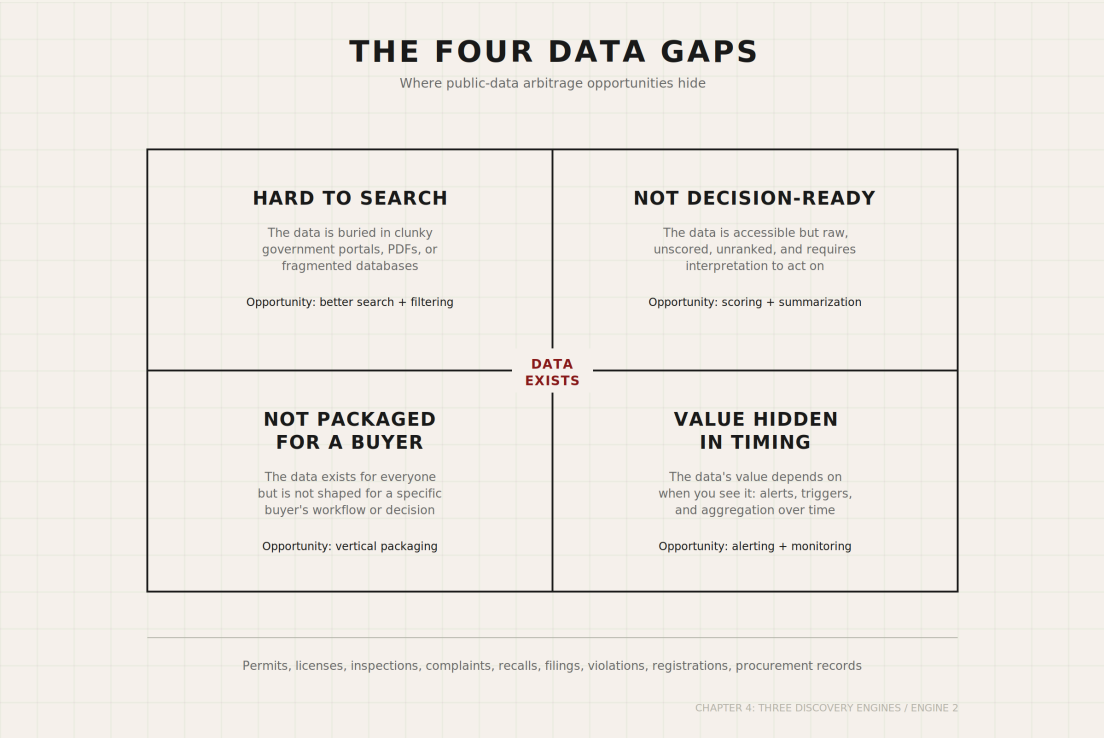
You are looking for evidence, not vibes.

Engine 2: Public-Data Arbitrage

This engine works when valuable data exists but the interface is broken, the SEO is weak, or the commercial use case is hidden.[13]

[12] Stop asking for ideas. Start running engines.

[13] Look for: permits, licenses, inspections, complaints, recalls, filings, violations, registrations, procurement records.



Engine 3: Workflow Compression

Some markets still run on spreadsheets, copy-paste lookups, scattered vendor tabs, ad hoc checklists, inbox-and-memory project management, and brittle manual processes.[14]

That is fertile ground.

Most strong solo-builder products come from one engine or a combination of two. The point is to stop asking for ideas and start running engines.

[14] A workflow-compression product does not need to be revolutionary. It needs to reduce steps, reduce mistakes, and save time.

CHAPTER 5

Kill Signals

A lot of progress comes from saying no faster.

Bad ideas do not fail quietly. They leak weakness early.[15]

The buyer is blurry

If you cannot say exactly who this is for, the problem is not ready.

It smells like a feature, not a product

If it only works as an add-on to a larger system and has no standalone buying trigger, it may not deserve to exist on its own.

The path to money is hand-wavy

[15] The strongest founders are not the ones who discover the most opportunities. They are the ones who kill weak ones fastest.

“Maybe ads,” “maybe marketplace later,” and “maybe enterprise eventually” are not real monetization plans.

The pain is polite, not painful

If there is no urgency, cost, embarrassment, or operational consequence, move on.

Distribution is wishful

If the distribution thesis is mostly hope, the opportunity is weaker than it looks.

The wedge is too broad

If the first version tries to serve everyone in a category, you are not choosing a wedge. You are avoiding the discipline of choosing.[16]

You cannot reach the buyer

A problem can be real and still be a bad bet if the buyer is behind trust barriers, long procurement cycles, or domain constraints you do not want to fight.

The product needs too much before it can prove anything

If the wedge requires many integrations, polished UX, or heavy infrastructure before it can validate, that is a warning.

It is legally contaminated

Some opportunities are real but not worth the exposure.

You are excited mainly because the tooling is fun

If the best argument for the idea is that the stack is interesting, the market is probably not the real driver.

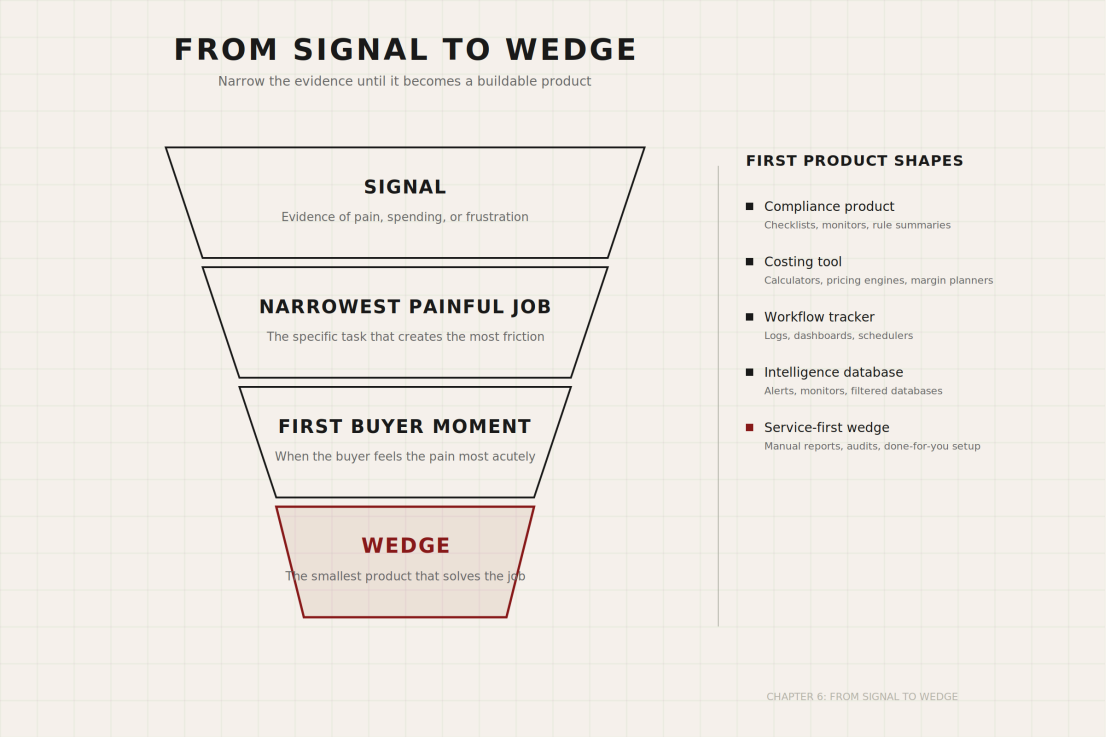
[16] Two or more kill signals checked = kill or reshape the wedge.

CHAPTER 6

From Signal to Wedge

A pain signal is not a product. It is evidence.[17]

Your job is to turn that evidence into a wedge narrow enough to validate and strong enough to matter.

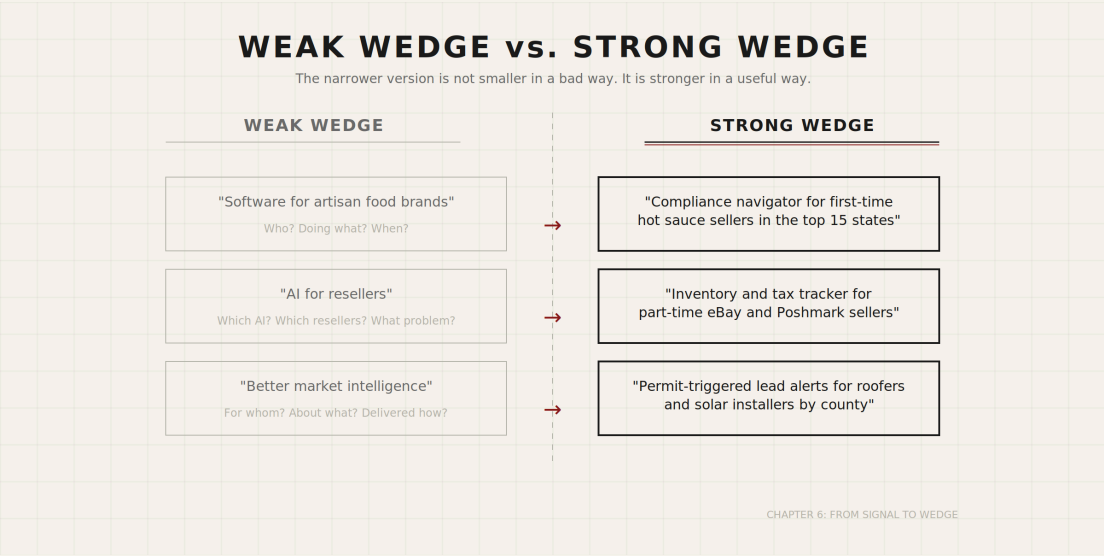


Common first product shapes include compliance products (checklists, monitors, rule summaries), costing tools (calculators, pricing engines, margin planners), workflow trackers (logs, dashboards, schedulers), searchable intelligence databases (alerts, monitors, filtered databases), and service-first wedges (manual reports, audits, done-for-you setup).

The Wedge Test

A good wedge usually has one obvious buyer, one painful job, one trigger moment, one clear promise, and one believable first version.

[17] Signal → narrowest painful job → first buyer moment → smallest product shape.

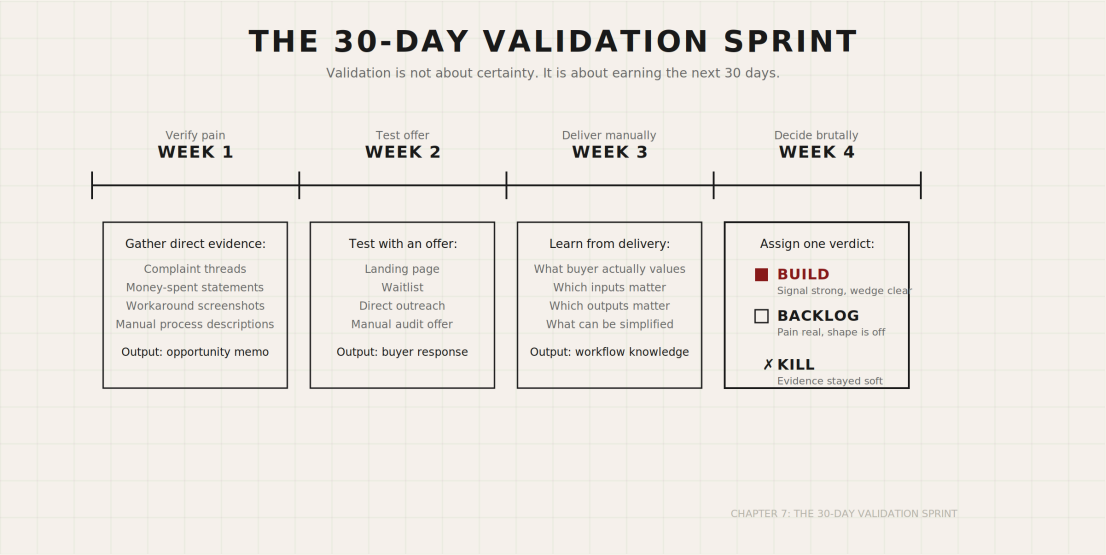


The narrower version is not smaller in a bad way. It is stronger in a useful way.

CHAPTER 7

The 30-Day Validation Sprint

Validation is not about certainty. It is about earning the next 30 days.[18]



Week 1: Verify the Pain

Gather direct evidence: complaint threads, money-spent statements, workaround screenshots, manual process descriptions, compliance confusion, repeated buying triggers. Then write a one-page opportunity memo.

Week 2: Test the Offer

You do not need a product yet. You need an offer. Test it with a landing page, waitlist, direct outreach, a manual audit offer, or a concierge service.

Week 3: Deliver Manually

This is where many founders cheat. They collect emails and call that validation. Usually it is not.[19]

A concrete example: if you are testing permit-triggered lead alerts, you do not need a dashboard first. You can manually pull filings for one county, clean them in a spreadsheet, and send a weekly

[18] Four questions: Is the pain real? Is the buyer reachable? Will anyone pay? Is the wedge small enough to ship?
[19] If no one wants the manual result, software will not save you.

alert email to three contractors. If they reply, ask questions, or ask you to keep sending it, you have signal.

Week 4: Decide Brutally

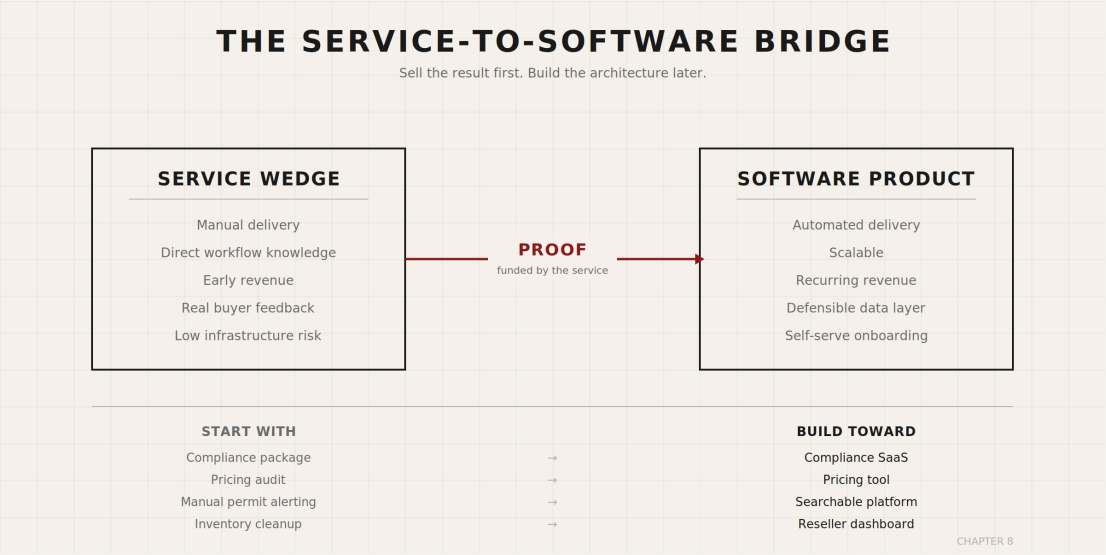
Then assign one verdict: **BUILD** if the signal is strong and the wedge is clear. **BACKLOG** if the pain is real but timing, access, or shape is off. **KILL** if the evidence stayed soft or the build path is too heavy.

CHAPTER 8

First Customer Before Full Product

Most solo founders overbuild because software feels like progress. Often it is avoidance.[20]

The fastest way to learn whether a wedge matters is to get a first customer before the product is fully formed. One of the cleanest moves for solo builders is to start with a service-shaped wedge before converting it into software.



This does three things: gets you closer to money faster, gives you direct workflow knowledge, and funds the build instead of waiting for the build to fund itself.

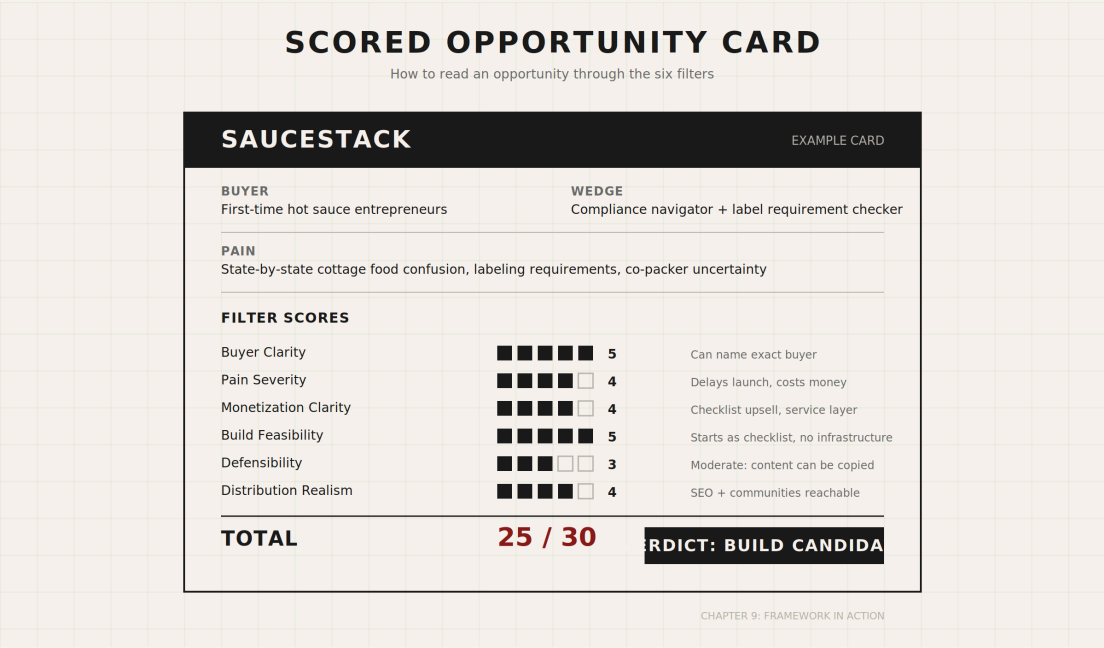
[20] Sell the result. The buyer cares about the outcome, not the architecture.

CHAPTER 9

Framework in Action

The point of this section is not volume. It is proof.[21]

These are the kinds of wedges the framework tends to surface when the filters are doing their job.



1. SauceStack

Buyer: First-time hot sauce entrepreneurs. **Pain:** State-by-state cottage food confusion, labeling requirements, and co-packer uncertainty. **Wedge:** Compliance navigator plus label requirement checker.[22]

Why it works: Clear buyer, urgent trigger, and obvious willingness to pay because mistakes delay launch. **What the framework sees:** Visible regulatory pain, a strong first-customer moment, and a narrow entry wedge that can begin as a checklist before software deepens.

2. FlipTrack

[21] These are not recommendations. They are teaching examples showing what strong opportunity structure looks like.

[22] SauceStack scores 25/30. Buyer clarity and build feasibility are perfect 5s. Defensibility is the weak point at 3.

Buyer: Part-time and full-time resellers. **Pain:** Inventory chaos, tax-season panic, unclear profit by item or platform. **Wedge:** Cross-platform inventory plus tax-ready export.

Why it works: Repetitive pain, recurring use, measurable value. **What the framework sees:** Ugly repeated workflow, obvious cost of mistakes, and a manual bridge that can start with cleanup plus reporting.

3. Building Permit Cross-Sell Intelligence

Buyer: Roofers, solar installers, remodelers, and adjacent home-service operators. **Pain:** Weak visibility into new jobs at the right time. **Wedge:** Permit-triggered lead alerts by trade and geography.[23]

Why it works: The timing value is easy to explain and easy to sell. **What the framework sees:** Public-data arbitrage plus strong distribution realism.

4. Federal Contracting Opportunity Engine

Buyer: Government contractors, consultants, and capture teams. **Pain:** Incumbent tools are expensive and government interfaces are painful. **Wedge:** Better filtering, search, and alerting around solicitations.

Why it works: Clear spend, ugly existing experience, high value per win.

5. CraftLedger

Buyer: Leather crafters and commission-based makers. **Pain:** Underpricing labor, weak quoting discipline, poor materials tracking. **Wedge:** Project costing and pricing calculator.

Why it works: Small pricing improvements create immediate ROI. **What the framework sees:** A narrow buyer, recurring pain, and a wedge that can be sold as a tool, worksheet, or service before it becomes software.

The common pattern should be obvious by now. The strongest opportunities are usually not the most glamorous. They are the ones where pain is visible, the trigger is real, and the first version can solve one painful job without pretending to be a whole company on day one.

[23] Permit data is the classic public-data arbitrage play: the data exists, the interface is terrible, and the timing value is obvious to the buyer.

CHAPTER 10

What to Build Next

The final decision is not, “What sounds best?”

It is, “What deserves the next 30 days more than the alternatives?”

Choose the opportunity that best combines strongest score, clearest wedge, fastest path to proof, best fit with your actual strengths, and lowest risk of dragging you into a swamp before first revenue.

The best next build is often not the biggest market. It is the cleanest entry point.[24]

Do not let two promising ideas advance at the same time unless one is clearly a sidecar asset.

Build one. Backlog one. Kill the rest.

That is enough.

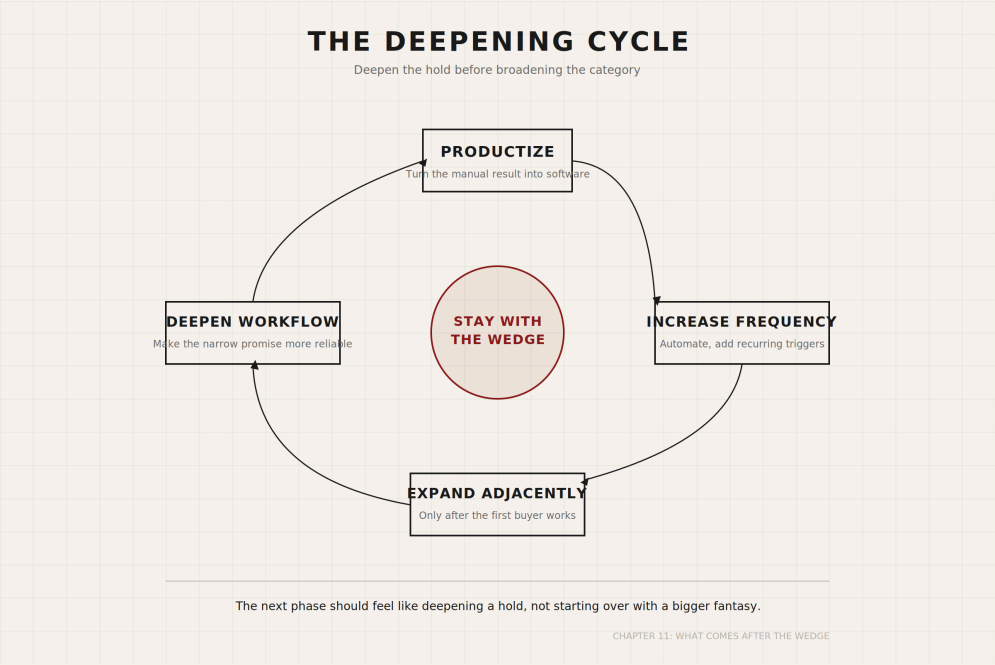
[24] Parallel idea pursuit feels productive. Usually it is focus leakage.

CHAPTER 11

What Comes After the Wedge

If the wedge works, the next job is not to explode the scope. It is to deepen the proof.[25]

That usually means one of four moves: productize the manual result, increase frequency or automation, expand to adjacent buyers only after the first buyer is working, and deepen the workflow before broadening the category.



This is where a lot of builders relapse. They survive the idea trap, prove a narrow wedge, then immediately try to turn it into a broad platform because the early signal feels like permission to expand.

It usually is not.

The wedge earned the next phase because it was specific. Keep respecting that. Stay with the buyer. Stay with the trigger. Stay with the painful job. Make the recurring version better before you start imagining the category diagram.

The next phase should feel like deepening a hold, not starting over with a bigger fantasy.

[25] The four post-wedge moves: productize, increase frequency, expand adjacently, deepen workflow.

Conclusion

This framework sounds ruthless because it is.

That is also why it is useful.

The hard part is not understanding the filters. The hard part is using them against ideas you actually like. Killing weak ideas is emotionally expensive because weak ideas are rarely worthless. They are usually just mistimed, under-shaped, under-validated, or not strong enough to deserve the next few months of your life.[26]

That is what makes discipline difficult. You are not usually cutting obvious trash. You are cutting things that could maybe become something later. But “maybe later” is exactly how builders get trapped in soft conviction and scattered attention.

This book is not here to make you feel like you have more options.

It is here to make you choose better.

You do not need a hundred ideas. You need one strong wedge, one real buyer, one believable path to money, and the discipline to ignore everything else until the market answers you.

That is enough to build something real.

[26] You are not usually cutting obvious trash. You are cutting things that could maybe become something later.

Appendices

APPENDIX

Pattern Library

These are shorter opportunity patterns worth studying. They are pattern exercises, not the main proof section.

HiveStack

Buyer: Sideline beekeepers. **Pain:** Colony loss, treatment timing, weak profitability tracking. **Wedge:** Hive log plus treatment scheduler plus simple economics tracker.

Liquor License Opening Alerts

Buyer: Restaurant vendors, equipment suppliers, POS sellers. **Pain:** Missing the earliest buying window. **Wedge:** License approval alert feed.

CFPB Complaint Intelligence

Buyer: Consumers, analysts, financial marketers. **Pain:** Complaint data exists but is hard to interpret. **Wedge:** Searchable complaint trends and reputation monitoring.

Restaurant Inspection Crisis Alerts

Buyer: Restaurant operators and reputation managers. **Pain:** Inspection failures damage revenue fast. **Wedge:** Alerting plus rapid-response checklist.

Contractor License Violation Monitor

Buyer: Contractors, insurers, sureties. **Pain:** Licensing lapses are hard to track across jurisdictions. **Wedge:** Monitoring plus alerting plus referral hooks.

Small-Batch Production Tracker

Buyer: Artisan food and handmade-goods producers. **Pain:** Spreadsheet-heavy batch logs, weak costing. **Wedge:** Batch management for one production type first.

UCC Filing Lead Intelligence

Buyer: Insurers, lenders, equipment vendors. **Pain:** Weak visibility into newly financed businesses. **Wedge:** Financing event trigger intelligence.

Compliance Launch Planner

Buyer: Niche operators moving from hobby to business. **Pain:** No structured path across entity setup, compliance, and first sales. **Wedge:** Vertical-specific launch planner with templates.

APPENDIX

Opportunity Scorecard

Rate each idea from **1 to 5** on: buyer clarity, pain severity, monetization clarity, build feasibility, defensibility, and distribution realism.

Interpretation: 26–30 = strong build candidate. 21–25 = good candidate, pressure-test first. 16–20 = backlog or tighten the wedge. 15 and below = kill unless new evidence appears.

APPENDIX

Opportunity Worksheet

For every serious opportunity, fill this out before building:

Working name .....

Buyer .....

Pain .....

Current workaround .....

Trigger moment .....

Wedge .....

Why now .....

Path to money .....

Fastest way to test .....

Main risk .....

Build / Backlog / Kill .....

If you cannot complete this in plain English, the idea is still too fuzzy.